

Estruturas de Dados em C++

Vetores e Matrizes aplicados ao conceito de CRUD

1. Introdução

Em programação, muitas vezes precisamos **armazenar vários dados do mesmo tipo**.

Para isso utilizamos estruturas chamadas **Vetores e Matrizes**.

Essas estruturas permitem realizar operações semelhantes às usadas em bancos de dados, como:

- **Cadastrar** dados
- **Consultar** dados
- **Modificar** dados
- **Excluir** dados

Esse conjunto de operações é conhecido como **CRUD**, sigla em inglês para:

- **Create** (Cadastrar)
- **Read** (Consultar)
- **Update** (Modificar)
- **Delete** (Excluir)

Mesmo sem utilizar banco de dados, podemos implementar essas operações usando **vetores ou matrizes na memória do programa**.

2. Conceito de Vetor

Um **vetor** é uma estrutura que armazena **vários valores do mesmo tipo** em posições sequenciais de memória.

Cada posição é identificada por um **índice**, que começa em **0**.

Exemplo:

Índice: 0 1 2 3 4

Valor : 10 20 30 40 50

Declaração em C++:

```
int vetor[5];
```

Isso cria um vetor com **5 posições**.

3. Exemplo prático: Cadastro de alunos usando vetor

Neste exemplo, utilizaremos um vetor para **armazenar notas de alunos**.

```
#include <iostream>
using namespace std;

int main(){

    int notas[5];

    for(int i=0;i<5;i++){
        cout << "Digite a nota do aluno " << i+1 << ": ";
        cin >> notas[i];
    }

    cout << "\nNotas cadastradas:\n";

    for(int i=0;i<5;i++){
        cout << notas[i] << endl;
    }

}
```

Aqui estamos realizando a operação **Cadastrar (Create)**.

4. CRUD usando Vetores

4.1 Cadastrar (Create)

Cadastrar significa **inserir novos dados no vetor**.

Exemplo:

```
for(int i=0;i<5;i++){
    cout << "Digite o valor: ";
    cin >> vetor[i];
}
```

4.2 Consultar (Read)

Consultar significa **buscar ou visualizar dados armazenados**.

Exemplo: buscar um número.

```
int valor;

cout << "Digite o valor a buscar: ";
cin >> valor;

for(int i=0;i<5;i++){
```

```
    if(vetor[i] == valor){  
        cout << "Valor encontrado na posicao " << i << endl;  
    }  
}
```

4.3 Modificar (Update)

Modificar significa **alterar um valor existente**.

Exemplo:

```
int posicao;  
  
cout << "Qual posicao deseja modificar? ";  
cin >> posicao;  
  
cout << "Digite o novo valor: ";  
cin >> vetor[posicao];
```

4.4 Excluir (Delete)

Excluir em vetor geralmente significa **zerar ou marcar a posição como vazia**.

Exemplo:

```
int posicao;  
  
cout << "Digite a posicao para excluir: ";  
cin >> posicao;  
  
vetor[posicao] = 0;
```

Em sistemas maiores, usa-se um valor especial como **-1 para indicar posição vazia**.

5. Conceito de Matriz

Uma **matriz** é uma estrutura bidimensional composta por **linhas e colunas**.

Exemplo de matriz 3×3:

```
    Colunas  
    0  1  2  
  
0 | 5  8  2
```

```
1 | 1 7 9
2 | 4 3 6
```

Acessamos elementos assim:

`M[linha][coluna]`

Exemplo:

`M[1][2] = 9`

Declaração em C++:

```
int matriz[3][3];
```

6. Aplicação prática: Cadastro de produtos com matriz

Podemos usar uma matriz para armazenar **vários dados de um produto**.

Exemplo:

Produto | Preço | Quantidade

Representação na matriz:

`produto[i][0] → código`

`produto[i][1] → preço`

`produto[i][2] → quantidade`

Declaração:

```
int produto[5][3];
```

Isso cria espaço para **5 produtos com 3 informações cada**.

7. CRUD usando Matrizes

7.1 Cadastrar produto

```
for(int i=0;i<5;i++){

    cout << "Codigo do produto: ";
    cin >> produto[i][0];

    cout << "Preco: ";
    cin >> produto[i][1];

    cout << "Quantidade: ";
    cin >> produto[i][2];

}
```

7.2 Consultar produto

```
int codigo;

cout << "Digite o codigo para buscar: ";
cin >> codigo;

for(int i=0;i<5;i++){

    if(produto[i][0] == codigo){

        cout << "Preco: " << produto[i][1] << endl;
        cout << "Quantidade: " << produto[i][2] << endl;

    }

}
```

7.3 Modificar produto

```
int codigo;

cout << "Digite o codigo para alterar: ";
cin >> codigo;

for(int i=0;i<5;i++){

    if(produto[i][0] == codigo){

        cout << "Novo preco: ";
        cin >> produto[i][1];

    }

}
```

7.4 Excluir produto

Podemos marcar o produto como removido.

```
int codigo;

cout << "Codigo para excluir: ";
cin >> codigo;
```

```

for(int i=0;i<5;i++){

    if(produto[i][0] == codigo){

        produto[i][0] = -1;
        produto[i][1] = 0;
        produto[i][2] = 0;

    }

}

```

8. Exemplo completo com menu (CRUD)

```

#include <iostream>
using namespace std;

int main(){

    int produto[5][3];
    int opcao;

    do{

        cout << "\n1 Cadastrar\n";
        cout << "2 Consultar\n";
        cout << "3 Alterar\n";
        cout << "4 Excluir\n";
        cout << "0 Sair\n";

        cin >> opcao;

    }while(opcao != 0);

}

```

Menus são muito usados em sistemas CRUD.

9. Comparação com Banco de Dados

Operação Banco de Dados Vetor/Matriz

Create	INSERT	Inserir no vetor
Read	SELECT	Buscar valor

Operação Banco de Dados Vetor/Matriz

Update	UPDATE	Alterar posição
Delete	DELETE	Limpar posição

10. Aplicações reais

Vetores e matrizes são usados em:

- Sistemas de cadastro
 - Jogos
 - Planilhas
 - Processamento de imagens
 - Inteligência artificial
 - Sistemas financeiros
-

A seguir está a **extensão do conteúdo anterior**, acrescentando o conceito de **persistência de dados**, ou seja, **salvar e carregar dados em arquivos externos**. Isso é muito importante para os alunos entenderem **como os sistemas guardam informações mesmo após o programa ser fechado**.

Esse conceito é chamado de **Persistência de Dados** e se aproxima do funcionamento de **bancos de dados**.

Persistência de Dados em C++

Salvando dados em arquivo externo

Quando utilizamos apenas **vetores ou matrizes**, os dados ficam armazenados **apenas na memória RAM**.

Isso significa que:

- quando o programa termina
- ou o computador é desligado

→ **todos os dados são perdidos.**

Para evitar isso utilizamos **arquivos externos**, onde os dados são gravados no disco.

Exemplos de arquivos:

- .txt

- .csv
- .dat

Biblioteca utilizada

Para manipular arquivos em C++ utilizamos a biblioteca:

```
#include <fstream>
```

Ela permite:

- criar arquivos
- salvar dados
- ler dados
- atualizar informações

Tipos de arquivos em C++

Escrita em arquivo

Classe utilizada:

ofstream

Significa:

output file stream

Serve para **escrever dados no arquivo**.

Leitura de arquivo

Classe utilizada:

ifstream

Significa:

input file stream

Serve para **ler dados do arquivo**.

Exemplo simples salvando vetor em arquivo

```
#include <iostream>
#include <fstream>
using namespace std;
```



```
int main(){

    int vetor[5];

    ofstream arquivo("dados.txt");

    for(int i=0;i<5;i++){

        cout << "Digite um valor: ";
        cin >> vetor[i];

        arquivo << vetor[i] << endl;

    }

    arquivo.close();

    cout << "Dados salvos no arquivo.";

}
```

Esse programa:

1. recebe valores
2. grava no vetor
3. salva no arquivo **dados.txt**

Lendo dados do arquivo

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){

    ifstream arquivo("dados.txt");

    int valor;

    while(arquivo >> valor){

        cout << valor << endl;

    }

    arquivo.close();

}
```

```
}
```

Esse código lê os dados salvos anteriormente.

Salvando dados de uma matriz

Exemplo salvando cadastro de produtos.

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){

    int produto[5][3];

    ofstream arquivo("produtos.txt");

    for(int i=0;i<5;i++){

        cout << "Codigo: ";
        cin >> produto[i][0];

        cout << "Preco: ";
        cin >> produto[i][1];

        cout << "Quantidade: ";
        cin >> produto[i][2];

        arquivo << produto[i][0] << " "
            << produto[i][1] << " "
            << produto[i][2] << endl;

    }

    arquivo.close();

}
```

O arquivo gerado ficará assim:

```
101 25 10
102 30 5
103 12 20
```

Lendo dados da matriz no arquivo

```
#include <iostream>
#include <fstream>
using namespace std;

int main(){

    int codigo, preco, quantidade;

    ifstream arquivo("produtos.txt");

    while(arquivo >> codigo >> preco >> quantidade){

        cout << "Codigo: " << codigo << endl;
        cout << "Preco: " << preco << endl;
        cout << "Quantidade: " << quantidade << endl;
        cout << endl;

    }

    arquivo.close();

}
```

CRUD com persistência de dados

Agora nosso sistema terá:

Operação Função

Cadastrar Inserir dados

Consultar Buscar dados

Modificar Alterar dados

Excluir Remover dados

Salvar Gravar em arquivo

Carregar Ler dados do arquivo

Exemplo de menu com salvar arquivo

```
#include <iostream>
#include <fstream>
using namespace std;
```

```
int main(){

    int vetor[5];
    int opcao;

    do{

        cout << "\n1 Cadastrar\n";
        cout << "2 Mostrar\n";
        cout << "3 Salvar em arquivo\n";
        cout << "0 Sair\n";

        cin >> opcao;

        if(opcao == 1){

            for(int i=0;i<5;i++){

                cout << "Digite valor: ";
                cin >> vetor[i];

            }

        }

        if(opcao == 2){

            for(int i=0;i<5;i++){

                cout << vetor[i] << endl;

            }

        }

        if(opcao == 3){

            ofstream arquivo("dados.txt");

            for(int i=0;i<5;i++){

                arquivo << vetor[i] << endl;

            }

            arquivo.close();

        }

    }

}
```

```
        cout << "Dados salvos.";

    }

    }while(opcao != 0);

}
```

Fluxo de funcionamento do sistema

Cadastrar dados
↓
Armazenar em vetor/matriz
↓
Salvar em arquivo
↓
Encerrar programa
↓
Abrir novamente
↓
Ler arquivo
↓
Recuperar dados

Conceito importante para os alunos

Esse processo simula **como sistemas reais funcionam**.

Exemplo:

Sistema	Armazenamento
Sistema escolar	Banco de dados
Sistema bancário	Banco de dados
Jogos	Arquivos de save
Aplicativos	Arquivos ou banco

Exercícios para alunos

Exercício 1

Criar um programa que:

- cadastre **10 números em um vetor**

- salve os números em um arquivo
-

Exercício 2

Criar um sistema que:

- cadastre **5 produtos**
 - salve os produtos em arquivo
 - leia os produtos do arquivo
-

Exercício 3

Criar um menu com:

- 1 Cadastrar aluno
- 2 Consultar aluno
- 3 Alterar aluno
- 4 Excluir aluno
- 5 Salvar em arquivo
- 6 Carregar arquivo

Exercício de processual:

Trabalhando com Vetores e Matrizes

Objetivo: Aplicar no tema de projeto inicial estes conceitos abaixo:

Aplicar os conceitos de:

- Vetores
 - Matrizes
 - Funções
 - CRUD (Cadastrar, Consultar, Modificar, Excluir)
-

Cadastro de Notas com Vetor

Crie um programa que utilize um **vetor para armazenar as notas de 10 alunos**.

Declaração do vetor:

```
float notas[10];
```

O programa deve permitir:

1. Cadastrar notas

2. Consultar notas
 3. Modificar uma nota
 4. Excluir uma nota
 5. Calcular média da turma
 6. Sair
-

Estrutura do Menu

```
cout << "1 - Cadastrar notas\n";  
cout << "2 - Consultar notas\n";  
cout << "3 - Modificar nota\n";  
cout << "4 - Excluir nota\n";  
cout << "5 - Calcular media\n";  
cout << "0 - Sair\n";
```

Exemplo de Cadastro

```
for(int i = 0; i < 10; i++){  
    cout << "Digite a nota do aluno " << i+1 << ": ";  
    cin >> notas[i];  
}
```

Exemplo de Consulta

```
for(int i = 0; i < 10; i++){  
    cout << "Aluno " << i+1 << " nota: " << notas[i] << endl;  
}
```

Modificar Nota

```
int posicao;  
  
cout << "Qual aluno deseja modificar? ";  
cin >> posicao;  
  
cout << "Digite a nova nota: ";  
cin >> notas[posicao-1];
```

Excluir Nota

```
int posicao;  
  
cout << "Qual aluno deseja excluir a nota? ";  
cin >> posicao;  
  
notas[posicao-1] = 0;
```

Calcular Média

```
float soma = 0;  
  
for(int i=0;i<10;i++){  
    soma += notas[i];  
}  
  
float media = soma / 10;  
  
cout << "Media da turma: " << media << endl;
```

Exercício 13 – Cadastro de Produtos com Vetores (CRUD)

Agora vamos simular um **pequeno sistema de estoque** utilizando **vetores**.

Cada produto terá:

- Código
- Preço
- Quantidade

Declaração dos vetores:

```
int codigo[10];  
float preco[10];  
int quantidade[10];
```

Operações do Sistema

O programa deve possuir o menu:

- 1 - Cadastrar produto
- 2 - Consultar produto
- 3 - Alterar produto
- 4 - Excluir produto
- 5 - Listar produtos
- 0 - Sair

Cadastrar Produto

```
for(int i=0;i<10;i++){  
  
    cout << "Codigo: ";  
    cin >> codigo[i];  
  
    cout << "Preco: ";  
    cin >> preco[i];  
  
    cout << "Quantidade: ";  
    cin >> quantidade[i];  
  
}
```

Consultar Produto

Buscar produto pelo código.

```
int busca;  
  
cout << "Digite o codigo: ";  
cin >> busca;  
  
for(int i=0;i<10;i++){  
  
    if(codigo[i] == busca){  
  
        cout << "Preco: " << preco[i] << endl;  
        cout << "Quantidade: " << quantidade[i] << endl;  
  
    }  
  
}
```

Alterar Produto

```
int busca;  
  
cout << "Codigo do produto: ";  
cin >> busca;  
  
for(int i=0;i<10;i++){  
  
    if(codigo[i] == busca){
```

```
        cout << "Novo preco: ";
        cin >> preco[i];

    }

}
```

Excluir Produto

```
int busca;

cout << "Codigo para excluir: ";
cin >> busca;

for(int i=0;i<10;i++){

    if(codigo[i] == busca){

        codigo[i] = -1;
        preco[i] = 0;
        quantidade[i] = 0;

    }

}
```

Exercício 14 – Cadastro com Matrizes

Agora vamos melhorar o sistema usando **matriz**, que permite armazenar vários dados do produto em uma única estrutura.

Declaração da matriz:

```
int produto[10][3];
```

Estrutura da Matriz

	Linha	Código	Preço	Quantidade
0	produto[0][0]	produto[0][1]	produto[0][2]	
1	produto[1][0]	produto[1][1]	produto[1][2]	

Cadastrar Produto

```
for(int i=0;i<10;i++){  
  
    cout << "Codigo: ";  
    cin >> produto[i][0];  
  
    cout << "Preco: ";  
    cin >> produto[i][1];  
  
    cout << "Quantidade: ";  
    cin >> produto[i][2];  
  
}
```

Consultar Produto

```
int codigo;  
  
cout << "Digite o codigo: ";  
cin >> codigo;  
  
for(int i=0;i<10;i++){  
  
    if(produto[i][0] == codigo){  
  
        cout << "Preco: " << produto[i][1] << endl;  
        cout << "Quantidade: " << produto[i][2] << endl;  
  
    }  
  
}
```

Alterar Produto

```
int codigo;  
  
cout << "Codigo do produto: ";  
cin >> codigo;  
  
for(int i=0;i<10;i++){  
  
    if(produto[i][0] == codigo){  
  
        cout << "Novo preco: ";  
        cin >> produto[i][1];  
  
    }  
  
}
```

```
}  
  
}
```

Excluir Produto

```
int codigo;  
  
cout << "Codigo para excluir: ";  
cin >> codigo;  
  
for(int i=0;i<10;i++){  
  
    if(produto[i][0] == codigo){  
  
        produto[i][0] = -1;  
        produto[i][1] = 0;  
        produto[i][2] = 0;  
  
    }  
  
}
```

Exercício 15 – Desafio Final (Sistema Completo)

Criar um **sistema de estoque completo** com:

Menu:

- 1 - Cadastrar produto
- 2 - Consultar produto
- 3 - Alterar produto
- 4 - Excluir produto
- 5 - Listar produtos
- 6 - Salvar em arquivo
- 7 - Carregar arquivo
- 0 - Sair

Requisitos do Sistema

O programa deve utilizar:

- Vetores ou matriz
- Funções
- Estrutura de repetição

- Menu interativo
- Arquivos externos